



TeamFlow

A unified platform for software engineering teams to plan, execute, and investigate work in one place.

Contents

1. Project Overview & Setup
2. Domain Model / ERD
3. Architecture
4. Design Decisions Log

Systems Engineering Assignment · Design Document
Generated 2026-07-03

TeamFlow

A unified platform for software engineering teams to **plan, execute, and investigate** work in one place — replacing fragmented tools for project planning, task execution, incident investigation, notifications, and reporting.

TeamFlow brings multi-project organisation (Kanban / calendar / list views), task lifecycle management with dependencies, structured **Root Cause Analysis (RCA)** with a mandatory review sign-off, a single in-app + email notification pipeline with duplicate suppression, live analytics with CSV export, and a responsive light/dark interface.

This repository is the implementation deliverable for the TeamFlow Systems Engineering assignment. Written design deliverables live in [docs/](#) : [ERD](#) · [Architecture](#) · [Design Decisions](#).

Table of contents

- [Features implemented](#)
- [Tech stack](#)
- [Architecture at a glance](#)
- [Quick start](#)
- [Environment variables](#)
- [Demo accounts](#)
- [How the key business rules work](#)
- [Project structure](#)
- [API overview](#)
- [Assumptions](#)
- [Known limitations](#)
- [Future improvements](#)

Features implemented

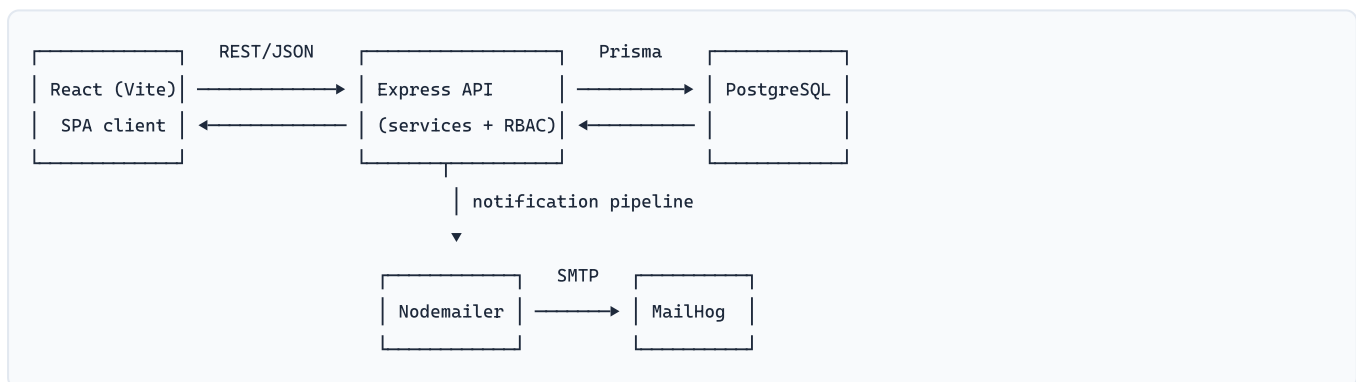
Area	What's built
Auth & users	JWT register/login, per-user theme + email opt-out preferences
Projects	Multi-project workspaces, membership with role-based access (OWNER/ADMIN/MEMBER/VIEWER)
Task views	Kanban (drag-and-drop), Calendar (by due date), List (sort + filter); view preference persisted per user, per project
Tasks	Priority, assignee, due/start dates, parent/sub-tasks, comments with @mention , file attachments
Dependencies	<code>BLOCKS</code> / <code>RELATES_TO</code> edges, cross-project capable, cycle detection ; dependency & assignee-overload conflicts surfaced as non-blocking warnings
Status workflow	Constrained status transitions (invalid moves rejected); every change written to an append-only activity log
RCA	Draft → submit → review lifecycle, four structured sections, severity, attachments, comments
Review sign-off	Multi-reviewer; an RCA cannot resolve until every assigned reviewer decides ; each decision requires a mandatory comment; any rejection ⇒ REJECTED

Area	What's built
Notifications	Single event pipeline → in-app bell + email ; log-before-dispatch ; duplicate suppression via a unique dedupe key; per-user email opt-out
Analytics	Live dashboard: completion rate, workload per assignee, velocity trend, RCA volume, project-health score
Export	CSV export of the task list scoped to the active filter/sort state
Theme & responsive	Light/dark toggle applied instantly (no reload); responsive layout from a shared component set
Audit	Append-only <code>ActivityLog</code> on every significant change, surfaced in an Activity feed

Tech stack

- **Frontend:** React 18 (Vite), React Router, Tailwind CSS
- **Backend:** Node.js + Express (REST API), Zod validation
- **Database:** PostgreSQL 16 via Prisma ORM (migrations + type-safe client)
- **Auth:** JWT (bcrypt password hashing)
- **Email:** Nodemailer → MailHog in development (captured, not sent)
- **File storage:** local disk adapter standing in for object storage (swappable for S3/GCS)
- **Infra (dev):** Docker Compose for Postgres + MailHog

Architecture at a glance



See [docs/architecture.md](#) for the full diagram, request lifecycle, and rationale (monolith vs microservices, SQL vs NoSQL, sync vs event-driven, file storage).

Quick start

Prerequisites: Node.js 18+ and Docker (for Postgres + MailHog). No local Postgres install needed.

```

# 1. Start Postgres + MailHog
docker compose up -d

# 2. Backend
cd server
cp .env.example .env
npm install
npx prisma migrate dev      # creates schema
npm run seed                # loads demo project, tasks, and an RCA
  
```

```

npm run dev                # API on http://localhost:4000

# 3. Frontend (in a second terminal)
cd client
npm install
npm run dev                # app on http://localhost:5173

```

Then open <http://localhost:5173> and sign in with a demo account below. Captured emails are viewable at <http://localhost:8025> (MailHog).

Windows note: Docker Desktop must be running before `docker compose up`. The DB is published on host port **5433** to avoid clashing with any local Postgres on 5432.

Without Docker

Point `DATABASE_URL` at any PostgreSQL instance and set the `SMTP_*` variables at a reachable mail server (or leave them — failed sends are recorded, not fatal). Then run the `server` steps above from step 2.

Environment variables

All backend config lives in `server/.env` (see [server/.env.example](#)):

Variable	Default	Purpose
<code>DATABASE_URL</code>	<code>postgresql://teamflow:teamflow@localhost:5433/teamflow</code>	Postgres connection string
<code>PORT</code>	<code>4000</code>	API port
<code>CLIENT_ORIGIN</code>	<code>http://localhost:5173</code>	Allowed CORS origin(s), comma-separated
<code>JWT_SECRET</code>	<code>change-me...</code>	Secret used to sign JWTs — set a long random value in production
<code>JWT_EXPIRES_IN</code>	<code>7d</code>	Token lifetime
<code>SMTP_HOST</code> / <code>SMTP_PORT</code>	<code>localhost</code> / <code>1025</code>	SMTP target (MailHog in dev)
<code>SMTP_SECURE</code> / <code>SMTP_USER</code> / <code>SMTP_PASS</code>	<code>false</code> / — / —	SMTP TLS + credentials
<code>EMAIL_FROM</code>	<code>TeamFlow <no-reply@teamflow.local></code>	From header on outbound mail
<code>UPLOAD_DIR</code>	<code>uploads</code>	Local object-storage directory
<code>MAX_UPLOAD_MB</code>	<code>10</code>	Max attachment size

The frontend needs no env vars in dev — Vite proxies `/api` to the backend (see `client/vite.config.js`).

Demo accounts

Seeded by `npm run seed` — all share the password `password123` :

Email	Name	Role in demo project
<code>alice@teamflow.dev</code>	Alice Owner	OWNER
<code>bob@teamflow.dev</code>	Bob Builder	MEMBER
<code>carol@teamflow.dev</code>	Carol Reviewer	ADMIN
<code>dave@teamflow.dev</code>	Dave Contributor	MEMBER

The seed includes a project (`TF`), five tasks with a dependency chain, a comment with an @mention, and an RCA in review with two assigned reviewers — enough to demo every flow immediately.

How the key business rules work

These are the behaviours the assignment calls out specifically; each is enforced server-side.

- **Dependency conflict** → When you add a `BLOCKS` edge, the server walks the existing graph and **rejects any edge that would create a cycle** (409). Softer conflicts (working a task that's still blocked, a due date earlier than a dependency's, an overloaded assignee) are returned as **warnings** and shown in the UI *without* blocking the save — matching the spec's "surface, don't block" stance.
- **Reviewer unavailable / review can't complete** → An RCA in review stays `IN_REVIEW` until **every** assigned reviewer submits a decision. If one reviewer never responds, the RCA simply does not resolve — it cannot be silently closed. The author can re-submit (reassigning reviewers) after a rejection. Every decision needs a mandatory comment, so no sign-off is undocumented.
- **Duplicate notifications** → Each alert is **logged before it is dispatched**. The log row carries a deterministic `dedupeKey` = `hash(recipient, type, entity, channel, 60s-bucket)` with a **unique constraint**. A replayed or double-fired event computes the same key, the insert is rejected, and the notification is marked `SUPPRESSED` instead of being sent twice.
- **Constrained task progression** → Status changes must follow an allowed transition map; invalid jumps return 400. Every accepted change is appended to the `ActivityLog`, so project history is trustworthy.

Project structure

```
teamflow/
├─ docker-compose.yml      # Postgres + MailHog
├─ docs/                   # ERD, architecture, design decisions
├─ server/                 # Express + Prisma API
│  └─ prisma/
│     ├── schema.prisma    # data model (source of truth)
│     ├── migrations/      # SQL migration history
│     └─ seed.js           # demo data
│  └─ src/
│     ├── app.js index.js  # app wiring + entry
│     ├── config.js prisma.js
│     ├── lib/             # auth, errors, mailer, csv, validate
│     ├── middleware/      # authenticate, project RBAC
│     ├── routes/          # auth, projects, tasks, rcas, attachments, notifications, analytics, users
│     └─ services/         # task, rca, notification, analytics, activity (business logic)
└─ client/                 # React + Vite + Tailwind
```

```

└─ src/
  └─ api.js ui.js
  └─ context/AppContext.jsx # auth + theme
  └─ pages/ # Login, Projects, Project, Settings
  └─ components/ # views/, TaskModal, RcaModal/Panel, Dashboard, ActivityFeed, NotificationBell

```

API overview

Base URL `http://localhost:4000/api`. All routes except `/auth/*` and `/health` require `Authorization: Bearer <token>`.

Method	Path	Purpose
POST	<code>/auth/register</code> , <code>/auth/login</code>	Obtain a JWT
GET/PATCH	<code>/auth/me</code>	Current user; update theme / email opt-out
GET/POST	<code>/projects</code>	List / create projects
GET/PATCH	<code>/projects/:id</code>	Project detail / update
POST/DELETE	<code>/projects/:id/members[:userId]</code>	Manage members
GET/PUT	<code>/projects/:id/view-preference</code>	Per-user view persistence
GET/POST	<code>/projects/:id/tasks</code>	List (filterable) / create tasks
GET/PATCH	<code>/tasks/:id</code>	Task detail / update fields
POST	<code>/tasks/:id/status</code>	Constrained status transition
POST/DELETE	<code>/tasks/:id/relations[:relId]</code>	Dependencies (cycle-checked)
GET/POST	<code>/tasks/:id/comments</code>	Comments with @mention
GET/POST	<code>/projects/:id/rcas</code> , <code>/rcas/:id</code>	RCA list / create / detail
PATCH	<code>/rcas/:id</code>	Edit sections (author)
POST	<code>/rcas/:id/submit</code>	Submit for review (assign reviewers)
POST	<code>/rcas/:id/reviews</code>	Record a decision (mandatory comment)
POST/GET	<code>/attachments</code> , <code>/attachments/:id</code>	Upload / download files
GET/POST	<code>/notifications</code> , <code>/notifications/mark-all-read</code>	In-app inbox
GET	<code>/projects/:id/dashboard</code>	Live analytics
GET	<code>/projects/:id/tasks/export.csv</code>	Filter-scoped CSV export
GET	<code>/projects/:id/activity</code>	Activity log feed

Assumptions

- **Single organisation / trusted tenancy.** There is no org boundary above projects; any user can be added to any project by an admin. Multi-tenant isolation is out of scope.
- **@mentions** are resolved by matching `@Full Name` against project members in the client; the server records the explicit `mentionIds` it receives.
- **Velocity** uses `DONE` tasks' `updatedAt` as a pragmatic completion proxy rather than a dedicated event store.

- **Attachments** are validated by size and MIME type only (no content scanning), and stored on local disk in dev as a stand-in for object storage.
- **Notifications** are polled by the client every 15s (no WebSocket), so in-app alerts have a short delay by design.
- **Auth** uses a bearer token in `localStorage` ; a production deployment would prefer httpOnly cookies + refresh tokens.

Known limitations

- In-app notifications have a **short delay** (15s poll) rather than appearing instantly.
- **Email failures surface** to the caller (recorded as `FAILED`) rather than being retried by a background worker.
- Uploaded files are **not content-inspected** beyond type and size.
- Dashboard aggregations read **live data**, so very large projects may load more slowly.
- The calendar can feel **crowded** when many tasks share a due date (capped to 3 chips + "+N more").
- Theme + auth token are **browser-bound** (localStorage); theme preference is also persisted server-side, but the token is not shared across devices.

Future improvements

Presence indicators, threaded comment replies, a drag-adjustable timeline/Gantt view, workload forecasting, recurring tasks, configurable multi-tier approval chains, background email retry with a real queue, WebSocket push for instant notifications, and cross-project trend analytics. See [docs/design-decisions.md](#) for how the current architecture leaves room for these.

Scripts reference

server/

Script	Action
<code>npm run dev</code>	Start API with autoreload
<code>npm run migrate:dev</code>	Create/apply a dev migration
<code>npm run migrate</code>	Apply migrations (deploy)
<code>npm run seed</code>	Load demo data
<code>npm run db:reset</code>	Drop, recreate, re-migrate, re-seed

client/

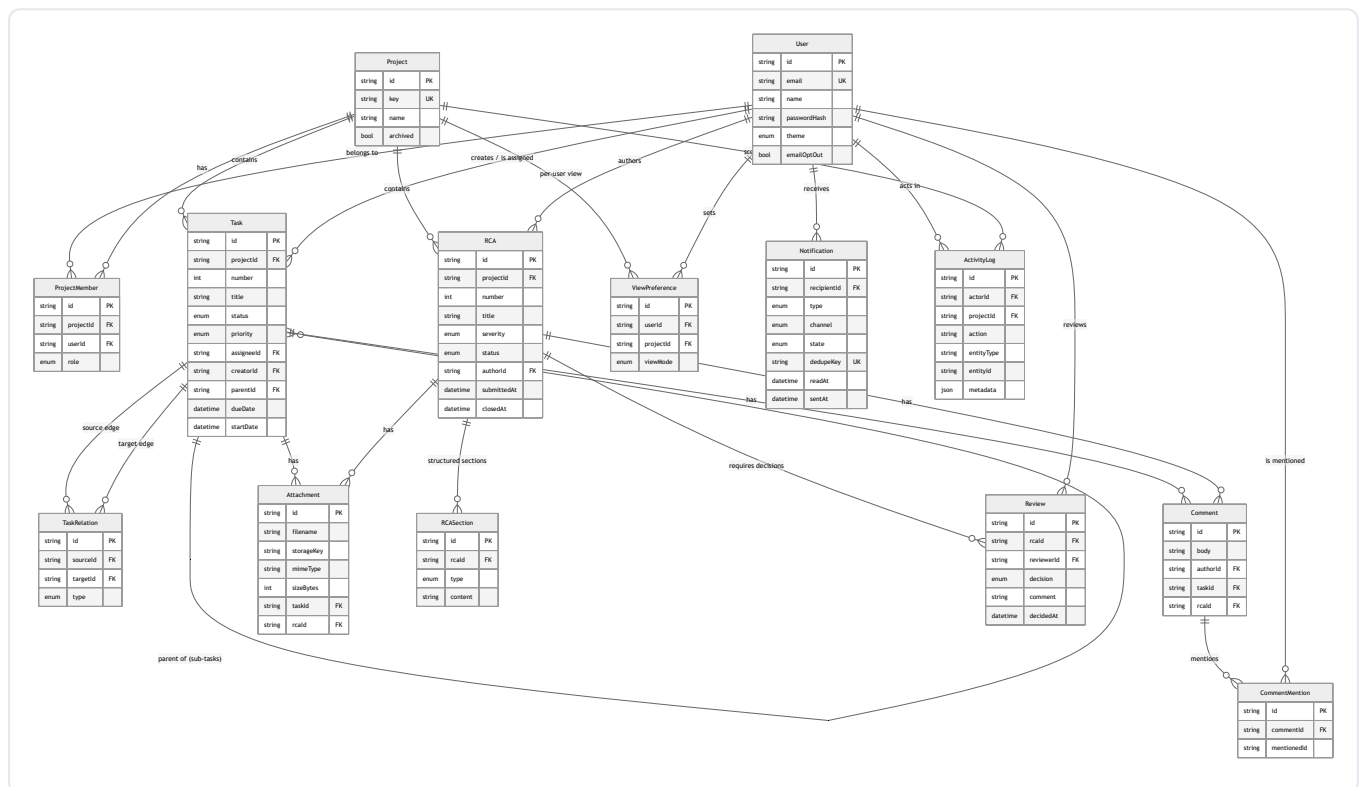
Script	Action
<code>npm run dev</code>	Vite dev server
<code>npm run build</code>	Production build
<code>npm run preview</code>	Preview the production build

TeamFlow — Domain Model / ERD

The data model is organised around **five primary aggregates**: Users, Projects, Tasks, RCAs, and Notifications. This document presents the entity-relationship diagram and explains the modelling decisions that matter — especially task dependencies and cross-project tracking.

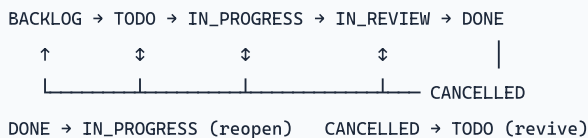
Source of truth: [server/prisma/schema.prisma](#) . SQL lives under [server/prisma/migrations/](#) .

Entity-relationship diagram



Lifecycle rules

Task status



Transitions are validated in `taskService.assertValidTransition` ; illegal jumps are rejected. Each accepted change appends an `ActivityLog` row.

RCA status



The transition out of **IN_REVIEW** is **derived** from the set of **Review** rows, not set directly — see `rcaService.deriveReviewOutcome`. It only resolves once **no review is PENDING**.

Key modelling decisions

1. Task dependencies as first-class edges (`TaskRelation`)

Dependencies are modelled as a **separate directional edge entity** rather than a self-referencing column on `Task`. A `BLOCKS` edge `source → target` means *target is blocked by source*.

- **Why an edge table:** a task can have many blockers and many dependents (many-to-many), which a single FK column cannot express. An edge table also lets us store the relation `type` (`BLOCKS` vs `RELATES_TO`) and attach metadata.
- **Cycle safety:** because `BLOCKS` edges form a dependency DAG, we run a reachability check before inserting an edge and reject anything that would create a cycle.
- **Uniqueness:** `@unique([sourceId, targetId, type])` prevents duplicate edges.

2. Cross-project dependency tracking

`TaskRelation` references two `Task` ids **without constraining them to the same project**. Two tasks in different projects can therefore be linked, satisfying the requirement that a task be *“tracked across more than one project.”* Access control is enforced at the API layer (the caller must be a member of *both* tasks’ projects to create the edge), keeping the data model flexible while authorisation stays explicit.

3. Sub-tasks via a self-relation

`Task.parentId → Task.id` (`TaskHierarchy`) gives a lightweight parent/child tree for breaking work down, distinct from the *dependency* graph above. Hierarchy and dependency are deliberately separate concerns.

4. Polymorphic comments & attachments

`Comment` and `Attachment` each carry **both** an optional `taskId` and `rcaId`, so the same collaboration primitives serve tasks and investigations from day one without duplicated tables. Application logic guarantees exactly one parent is set.

5. Reviews as pre-created slots

Submitting an RCA creates one `Review` row per assigned reviewer with `decision = PENDING`. Making each reviewer a **row** (unique per `(rcaId, reviewerId)`) is what lets the system know the *complete* set of required decisions — the sign-off rule (“cannot close until all reviewers decide”) is a simple check that no slot is still `PENDING`.

6. Notifications carry a unique `dedupeKey`

The `Notification` table doubles as the **delivery log** and the **duplicate-suppression mechanism**. Writing the row *before* dispatch, with a `UNIQUE dedupeKey`, turns “don’t send the same alert twice” into a database guarantee rather than application-level best-effort. `state` (`PENDING → SENT / SUPPRESSED / FAILED`) makes delivery auditable.

7. Append-only `ActivityLog`

Every significant state change writes an immutable log row (actor, action, entity, JSON metadata). This underpins the trustworthy project history the product stance depends on, and powers the Activity feed and audit needs without scattering history across tables.

8. Per-user, per-project `ViewPreference`

A dedicated table keyed `@@unique([userId, projectId])` stores whether a user prefers Kanban / calendar / list for each project, so the choice persists across sessions and devices without polluting the `Project` or `User` rows.

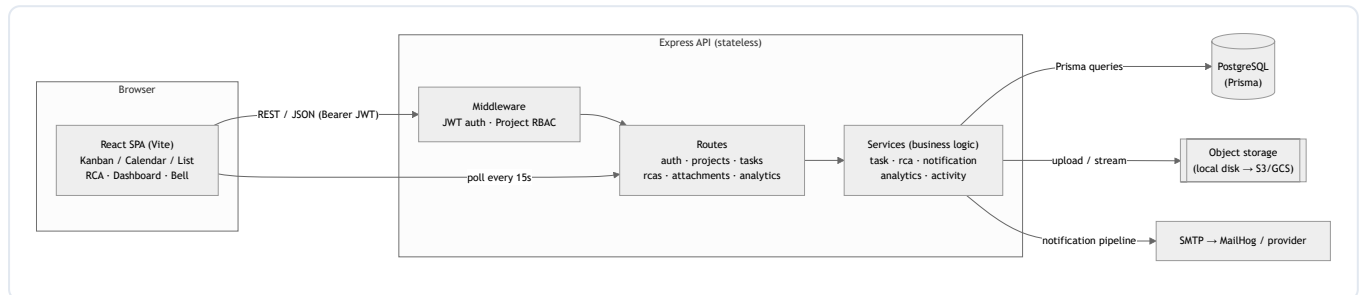
Constraints summary

Constraint	Where	Guarantees
<code>User.email</code> unique	schema	one account per email
<code>Project.key</code> unique	schema	stable human-readable task keys (<code>TF-14</code>)
<code>ProjectMember (projectId,userId)</code> unique	schema	one role per user per project
<code>Task (projectId, number)</code> unique	schema	sequential per-project numbering
<code>TaskRelation (sourceId,targetId,type)</code> unique	schema	no duplicate edges
No dependency cycles	<code>taskService</code>	acyclic <code>BLOCKS</code> graph
<code>Review (rcaId,reviewerId)</code> unique	schema	one decision slot per reviewer
<code>RCASection (rcaId,type)</code> unique	schema	one of each structured section
<code>Notification.dedupeKey</code> unique	schema	duplicate alerts suppressed
Valid status transitions	<code>taskService</code>	trustworthy task history
RCA resolves only when all reviews decided	<code>rcaService</code>	documented sign-off

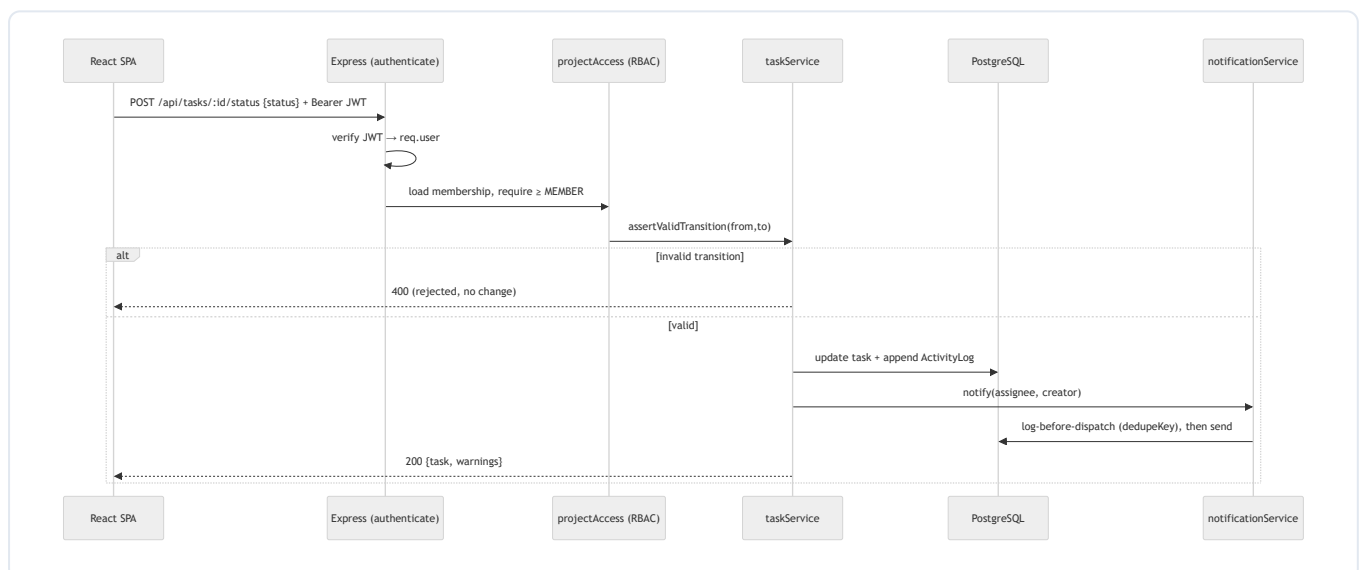
TeamFlow — Architecture

1. System overview

TeamFlow is a **client-server web application**: a React single-page app talking to a stateless Express REST API backed by PostgreSQL, with an internal notification pipeline that fans out to in-app storage and email.



Request lifecycle (example: change task status)



2. Architectural decisions & justification

Monolith vs microservices → **Modular monolith**

A single deployable API with **clear internal module boundaries** (`routes/` for transport, `services/` for business logic). For a team-scale product with tightly-coupled aggregates (tasks, RCAs, notifications all reference the same users/projects), a monolith gives transactional consistency, one migration story, and trivial local setup. The service layer is already split by domain, so any hot module (e.g. notifications) can be extracted into its own service later **without rewriting callers**. Microservices now would add network hops, distributed transactions, and ops overhead we don't need at this scale.

SQL vs NoSQL → **PostgreSQL (relational)**

The domain is **highly relational and constraint-heavy**: unique keys, foreign keys, many-to-many dependency edges, "resolve only when all reviewers decided," per-project sequential numbering. These are naturally and safely expressed with relational integrity + transactions. A document store would push this consistency into application code and make cross-entity queries

(dashboards, dependency graphs) harder. Postgres also gives us JSON columns (`ActivityLog.metadata`) where we *do* want schemaless flexibility — best of both.

Synchronous vs event-driven → Synchronous request/response + an internal pipeline

API calls are synchronous REST so the client gets immediate, consistent results (a task move returns the updated task and any warnings). Notifications run through an **internal pipeline abstraction** (`notificationService.notify`) that is event-shaped — one `notify()` call fans out to multiple channels and records delivery state. This gives most of the decoupling benefit (single dispatch point, dedup, per-channel outcomes) **without** standing up a broker. The `notify()` boundary is exactly where a real message queue (SQS/Rabbit) would slot in for at-least-once delivery and background retry.

File storage → Object-storage abstraction, local disk in dev

Binary content is never stored in the database; `Attachment` holds a `storageKey` pointing at an object store. In development the store is the local filesystem (`uploads/`), isolated behind the attachments route. Swapping to S3/GCS means changing only that storage adapter — the schema, API, and client are unaffected. This keeps the DB small and lets file serving scale independently.

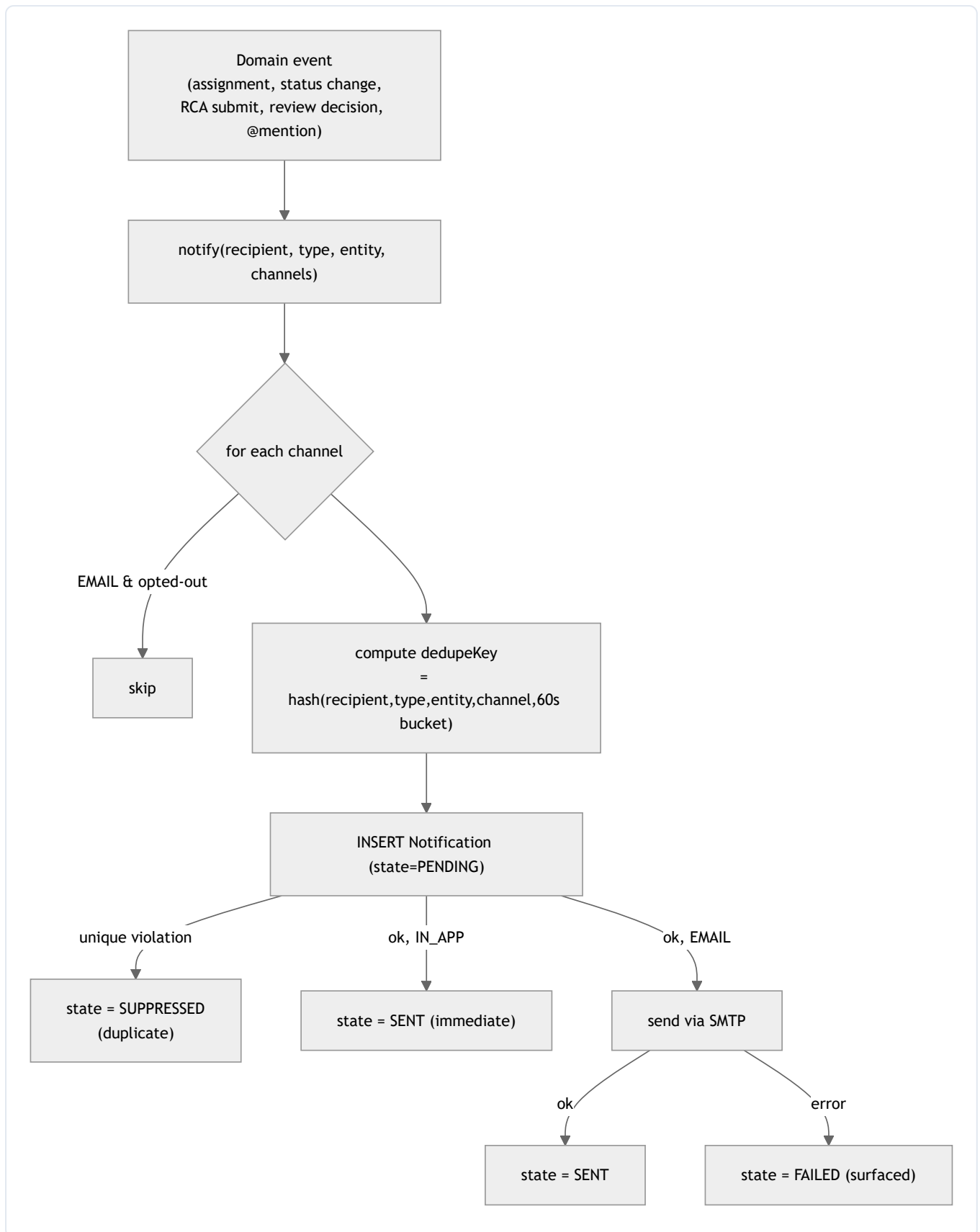
Authentication → Stateless JWT

The API keeps no server-side session, so it scales horizontally with no shared session store. Tokens are signed with `JWT_SECRET` and verified per request. Trade-off: revocation is coarse (short-ish expiry) — a production build would add refresh tokens and httpOnly cookies.

3. Cross-cutting concerns

Concern	Approach
AuthZ	Two-layer: <code>authenticate</code> (who you are) then <code>requireProjectRole</code> (what you may do in this project), ranked VIEWER < MEMBER < ADMIN < OWNER
Validation	Zod schemas at every write endpoint; failures return structured 400s
Error handling	Central Express error middleware maps typed <code>HttpError</code> s to responses; unknown errors → 500
Auditability	Append-only <code>ActivityLog</code> on every significant change
Consistency	Multi-write operations (e.g. RCA submit resets + creates reviews) run in Prisma transactions
Idempotent alerts	Log-before-dispatch + unique <code>dedupeKey</code>

4. Notification pipeline (detail)



The unique index on `dedupeKey` is the linchpin: even under concurrent or replayed events, the second insert fails and the alert is suppressed rather than duplicated — the exact scenario (“notification event triggered twice due to a processing lag”) the product is evaluated against.

5. Scaling & deployment notes

- **Stateless API** → run N replicas behind a load balancer; Postgres is the only stateful tier.
- **Read scaling** → dashboard aggregations are the heaviest reads; add read replicas and/or materialised summary tables if projects grow very large (current design reads live for correctness).
- **Notifications at scale** → replace the inline pipeline with a queue + worker for background email retry and burst absorption; the `notify()` contract stays the same.
- **Files at scale** → move `UPLOAD_DIR` to S3 and serve via signed URLs / CDN.
- **Multi-region** → Postgres primary + regional read replicas; object storage replicated per region; JWTs are region-agnostic. Offline-first would layer a client cache + sync log against the same REST API.

See [design-decisions.md](#) for the alternatives considered and trade-offs behind each choice.

TeamFlow — Design Decisions Log

Each decision records the **context**, the **alternatives considered**, the **trade-off**, and the **rationale**. They map to the product decisions and tradeoffs called out in the brief.

DD-1 · Modular monolith over microservices

- **Context:** One team-scale platform spanning planning, tasks, RCAs, notifications, reporting.
- **Alternatives:** (a) Microservices per aggregate; (b) serverless functions; © modular monolith.
- **Decision:** Modular monolith — one Express deployable, domain-split `services/`.
- **Trade-off:** Less independent scaling/deployment per module; but far simpler transactions, one migration path, and trivial local setup. The service boundaries make later extraction cheap.
- **Rationale:** Aggregates are tightly coupled through shared users/projects; premature service boundaries would add distributed-transaction pain with no scale need yet.

DD-2 · PostgreSQL over a document store

- **Context:** Constraint-heavy, relational domain (dependency edges, sign-off completeness, unique keys).
- **Alternatives:** MongoDB / DynamoDB; Postgres; Postgres + JSON hybrid.
- **Decision:** Postgres via Prisma, using JSON columns only where schemaless genuinely helps (`ActivityLog.metadata`).
- **Trade-off:** Rigid schema + migrations vs document flexibility; but we gain referential integrity and transactional guarantees for free.
- **Rationale:** The hardest rules (“resolve only when all reviewers decided”, cycle-free dependencies, per-project numbering) are relational integrity problems. Keeping them in the DB beats re-implementing them in app code.

DD-3 · Dependencies as a directional edge entity

- **Context:** Tasks depend on one another and must be trackable across projects.
- **Alternatives:** (a) `blockedById` FK column; (b) array column of ids; © `TaskRelation` edge table.
- **Decision:** Edge table with `type` (`BLOCKS` / `RELATES_TO`) and a uniqueness constraint.
- **Trade-off:** An extra join to read dependencies; but supports many-to-many, cross-project links, typed relations, and metadata.
- **Rationale:** A single FK can’t express many blockers/dependents, and an array can’t be validated or joined cleanly. The edge table also makes cycle detection a straightforward graph walk. **See DD-4.**

DD-4 · Reject dependency cycles, warn on softer conflicts

- **Context:** “How should the system behave when a dependency conflict arises?”
- **Alternatives:** (a) Block all conflicting saves; (b) allow everything, no feedback; © hard-block only true cycles, warn on the rest.
- **Decision:** A `BLOCKS` edge that would create a cycle is **rejected** (409). Working a still-blocked task, a due date before a dependency’s, or an overloaded assignee return **non-blocking warnings**.
- **Trade-off:** Warnings can be ignored; but hard-blocking soft conflicts would fight the team’s judgement and stall work.
- **Rationale:** A cycle is logically impossible to satisfy, so it must be prevented. Everything else is a *judgement* signal — the spec explicitly says surface conflicts and assignee overload “as warnings without blocking saves.”

DD-5 · Constrained task status transitions

- **Context:** “Task progression ... less ad-hoc flexibility, but project history stays trustworthy.”
- **Alternatives:** Free-form status editing vs an allowed-transition map.
- **Decision:** Explicit transition map; invalid jumps rejected; every change appended to `ActivityLog`.

- **Trade-off:** Users can't skip arbitrarily between states; but history becomes a reliable narrative.
- **Rationale:** Directly implements the product stance that a trustworthy shared history beats unconstrained flexibility.

DD-6 · RCA sign-off derived from reviewer slots

- **Context:** "An investigation cannot close until all assigned reviewers have decided"; behaviour when a reviewer is unavailable or review can't complete.
- **Alternatives:** (a) Single approver; (b) first-decision-wins; © one `Review` slot per reviewer, status derived from the full set.
- **Decision:** Pre-create a `PENDING Review` per reviewer; resolve only when none remain pending; any rejection ⇒ `REJECTED`; every decision needs a mandatory comment.
- **Trade-off:** Resolution can stall if a reviewer never responds; but no RCA is ever silently closed.
- **Rationale:** Modelling reviewers as rows makes "all decided" a trivial, race-free check. An unavailable reviewer simply leaves the RCA open (visible as "N reviewers outstanding") — the author can reject-and-resubmit with new reviewers if needed. Mandatory comments guarantee documented sign-off.

DD-7 · Log-before-dispatch notifications with a unique dedupe key

- **Context:** "Every alert is logged before dispatch, and duplicates are suppressed"; evaluation replays an event twice.
- **Alternatives:** (a) Fire-and-forget send; (b) in-memory dedup cache; © persist first with a unique `dedupeKey`.
- **Decision:** Insert the notification row (`PENDING`) *before* sending; `dedupeKey = hash(recipient, type, entity, channel, 60s bucket)` carries a `UNIQUE` constraint; a collision ⇒ `SUPPRESSED`.
- **Trade-off:** A slight delivery delay and one extra write per alert; but duplicate suppression becomes a **database guarantee**, and delivery state is auditable.
- **Rationale:** In-memory dedup fails across restarts and replicas. Persisting first is the only approach that survives concurrency, retries, and multiple API instances — and it doubles as the delivery log.

DD-8 · Single pipeline for in-app + email, with opt-out

- **Context:** "Alerts are delivered via an in-app bell and email from a single event pipeline."
- **Alternatives:** Separate code paths per channel vs one `notify()` fanning out to channels.
- **Decision:** One `notify()` entry point iterates channels; in-app is always delivered, email respects `User.emailOptOut`; each channel records its own outcome.
- **Trade-off:** A channel abstraction to maintain; but no divergence between how in-app and email alerts are decided or logged.
- **Rationale:** A single pipeline is what makes dedup, opt-out, and consistent logging uniform across channels — and where a queue slots in later.

DD-9 · Live analytics over pre-aggregation

- **Context:** "Dashboard figures reflect live data ... always current, but very large projects may see slower loads."
- **Alternatives:** (a) Cached/materialised rollups; (b) live aggregation at request time.
- **Decision:** Aggregate live in `analyticsService`.
- **Trade-off:** Slower on very large projects; but always correct with zero cache-invalidation complexity.
- **Rationale:** For current data volumes, correctness and simplicity win. Materialised summaries are a clean later optimisation (noted in architecture scaling) precisely because the aggregation logic is isolated in one service.

DD-10 · CSV export scoped to active filter state

- **Context:** "All list views support CSV export scoped to the active filter state."
- **Alternatives:** Export everything vs reuse the exact filter/sort the user is viewing.
- **Decision:** The export endpoint reuses the **same filter builder** (`buildTaskFilter`) as the list API; the client passes its current query params through.
- **Trade-off:** None significant; the export can never drift from the list because both share one filter function.

- **Rationale:** Guarantees “the exported file contained only filtered records, with nothing extra” by construction.

DD-11 · Object-storage abstraction for files

- **Context:** File storage strategy; files must not bloat the DB.
- **Alternatives:** (a) BLOBs in Postgres; (b) files behind a storage adapter (local disk now, S3 later).
- **Decision:** Store only a `storageKey` in the DB; binaries live behind the attachments route’s storage adapter.
- **Trade-off:** Local disk isn’t durable/shared across nodes in dev; but the swap to S3/GCS touches one module.
- **Rationale:** Keeps the DB lean and lets file serving scale independently, per the brief’s file-storage prompt.

DD-12 · Stateless JWT auth with two-layer authorisation

- **Context:** Users act across multiple projects with different roles.
- **Alternatives:** Server sessions vs stateless JWT; single global role vs per-project roles.
- **Decision:** Stateless JWT for identity + per-project `ProjectMember.role` (VIEWER<MEMBER<ADMIN<OWNER) enforced by middleware.
- **Trade-off:** Coarse token revocation; but horizontal scaling with no session store, and precise per-project permissions.
- **Rationale:** Authorisation is inherently project-scoped here, so identity (JWT) and permission (membership) are cleanly separated. Refresh tokens/httpOnly cookies are the noted production hardening.

DD-13 · Polling for in-app notifications (no WebSocket yet)

- **Context:** In-app bell needs to update; the brief accepts a short delay as a known limitation.
- **Alternatives:** WebSocket/SSE push vs periodic polling.
- **Decision:** Client polls every 15s.
- **Trade-off:** Up to ~15s latency; but zero extra infrastructure and simpler ops.
- **Rationale:** Matches the acknowledged limitation (“in-app notifications have a short delay”). Push is a clean upgrade behind the same data model when instant delivery is required.

Decision → requirement traceability

Requirement in brief	Decisions
Domain model incl. cross-project & dependent tasks	DD-3, DD-4
Architecture (monolith/SQL/sync/files)	DD-1, DD-2, DD-8, DD-11
Dependency conflict behaviour	DD-4
Reviewer unavailable / review can’t complete	DD-6
Notification behaviour (single pipeline, dedup, opt-out)	DD-7, DD-8, DD-13
Task workflow & trustworthy history	DD-5
Reporting & export	DD-9, DD-10
Permissions	DD-12